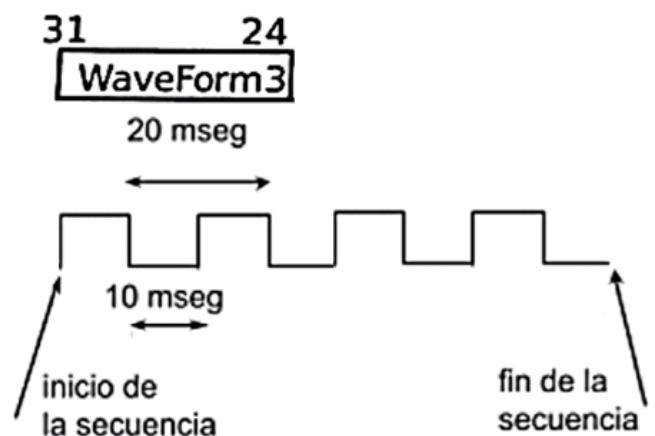
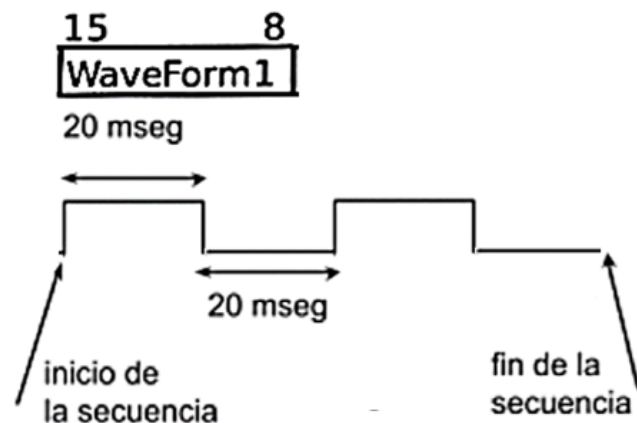
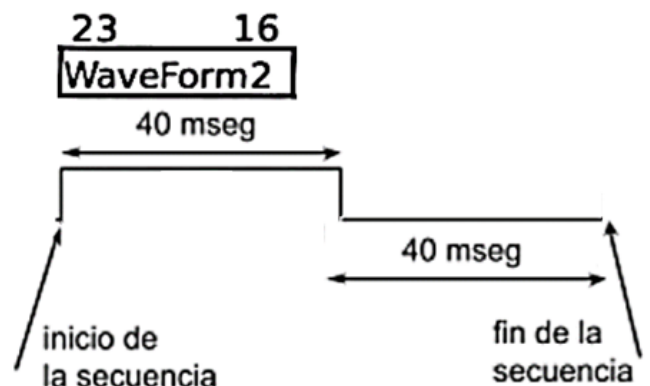
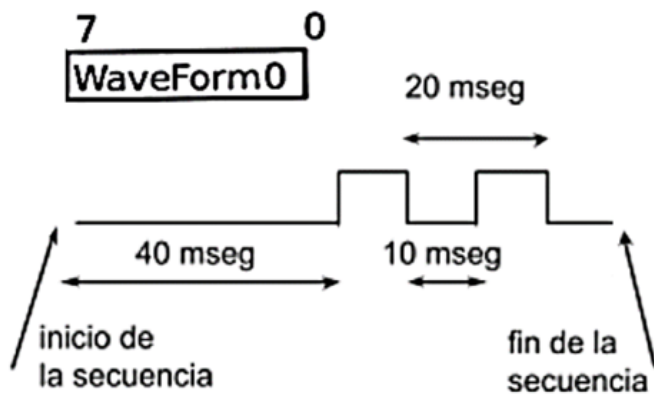
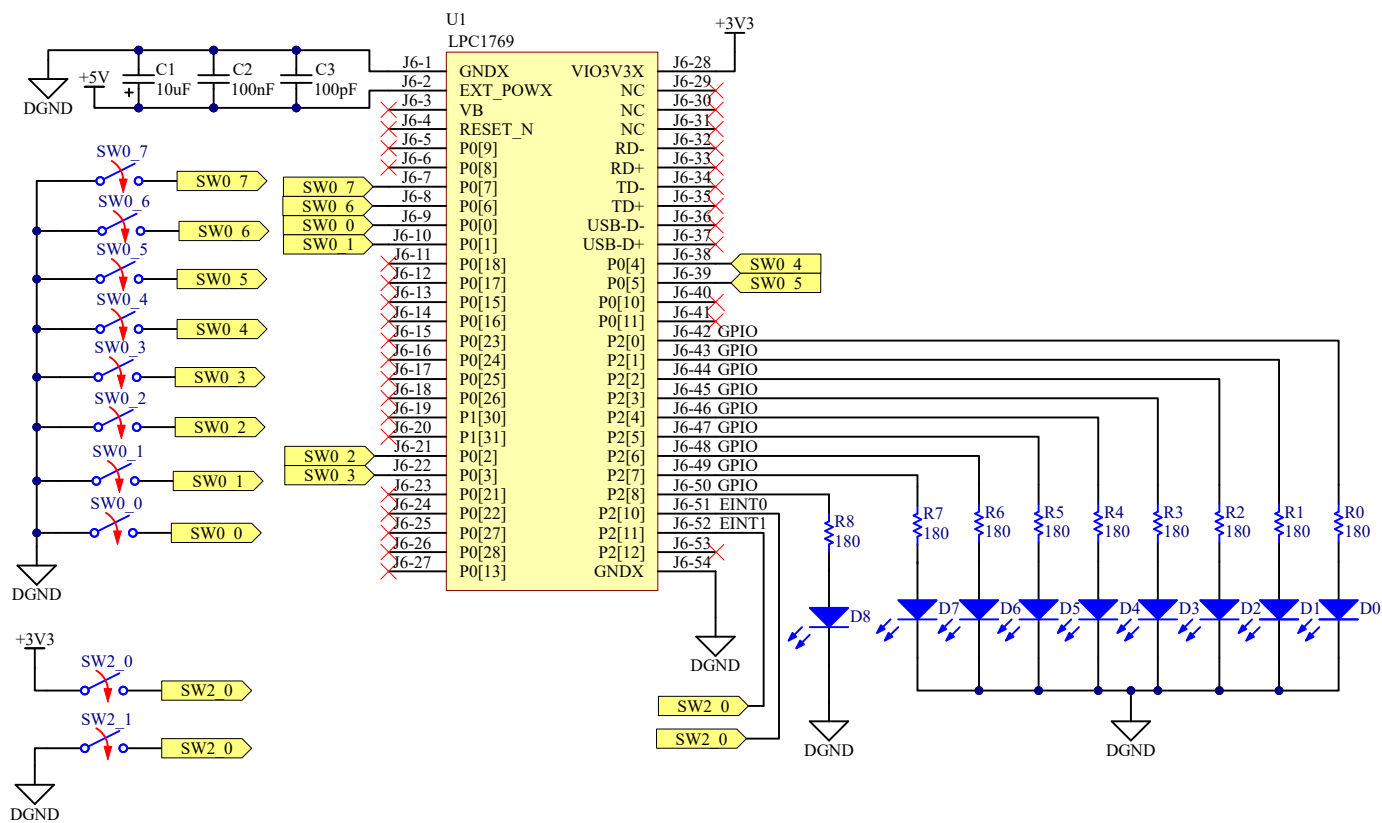


```

1  /*=====
2  * @file   Ejercicio_A_2.c
3  *
4  * @brief   Generador de formas de onda binarias serie por P2.8, visualización del
5  *          promedio en P2[7:0], control por interrupciones externas (EINT0, EINT1)
6  *          y máquina de estados por EINT3 para reconfiguración de patrones en RAM.
7  *
8  * @details - En la dirección de memoria 0x20080000 (RAM) se almacena un valor
9  *          de 32 bits que contiene cuatro formas de onda binarias de 8 bits
10 *          cada una.
11 *
12 *          31      24 23      16 15      8 7      0
13 *          ┌──────────┴──────────┴──────────┴──────────┐
14 *          │ WaveForm3 │ WaveForm2 │ WaveForm1 │ WaveForm0 │
15 *          └──────────┴──────────┴──────────┴──────────┘
16 *
17 *          Utilizando el temporizador SysTick y los registros del
18 *          microcontrolador LPC1769, generar por el pin P2.8 la señal binaria
19 *          serie correspondiente y desplegar por el puerto P2[7:0] el promedio
20 *          (conteo de bits activos) de la forma de onda seleccionada.
21 *
22 *          - Configurar una interrupción externa por EINT0 por flanco ascendente
23 *            y prioridad 3, para que al interrumpir finalice la transmisión de la
24 *            forma de onda actual y conmute a la siguiente (secuencia circular
25 *            de 0 a 3).
26 *
27 *          - Configurar una interrupción externa por EINT1 por flanco descendente
28 *            y prioridad 2, para alternar el período de la señal entre 80 ms
29 *            (por defecto) y 160 ms, considerando una frecuencia CCLK de 65 MHz.
30 *
31 *          - Configurar la interrupción por GPIO (asociada al vector EINT3) por
32 *            flanco descendente y prioridad 1 para ejecutar una máquina de 4 estados.
33 *            En cada interrupción, la máquina lee el patrón binario de 8 bits
34 *            presente en las entradas P0.0 a P0.7 y reescribe la forma de onda
35 *            correspondiente al estado actual (byte 0, 1, 2 o 3) en la memoria
36 *            0x20080000, avanzando secuencialmente al siguiente estado.
37 *
38 * @author   Ing. E. Migliore
39 *
40 * @version  3.0
41  */

```





```

1  /*=====
2  * DIRECTIVAS DE INCLUSIÓN
3  *=====*/
4  #include "LPC17xx.h"
5
6  /*=====
7  * DEFINICIÓN DE CONSTANTES Y MACROS
8  *=====*/
9  #define NVIC_PRIO_BITS      5U          /* Bits físicos de prioridad (0 al 31) */
10
11  /* Identificadores de Prioridad de IRQ */
12  #define SYSTICK_PRIORITY    //COMPLETAR
13  #define EINT3_PRIORITY      //COMPLETAR
14  #define EINT1_PRIORITY      //COMPLETAR
15  #define EINT0_PRIORITY      //COMPLETAR
16
17  /* Identificadores de IRQ */
18  #define SYSTICK_IRQ         //COMPLETAR
19  #define EINT0_IRQ           //COMPLETAR
20  #define EINT1_IRQ           //COMPLETAR
21  #define EINT3_IRQ           //COMPLETAR
22
23  /* Valores de recarga para SysTick */
24  #define N_TICKS_10MS        //COMPLETAR
25  #define N_TICKS_20MS        //COMPLETAR
26
27  #define SYSTICK_CTRL_ON      //COMPLETAR
28  #define SYSTICK_CTRL_OFF     //COMPLETAR
29
30  #define NUM_WAVES            4U
31  #define SIZE_WAVE            8U
32
33  /* Identificadores de forma de onda */
34  #define WAVE_FORM_0          0U
35  #define WAVE_FORM_1          1U
36  #define WAVE_FORM_2          2U
37  #define WAVE_FORM_3          3U
38
39  /* Patrones iniciales por defecto */
40  #define WAVE_FORM_0_PATTERN  0b00001010
41  #define WAVE_FORM_1_PATTERN  //COMPLETAR
42  #define WAVE_FORM_2_PATTERN  //COMPLETAR
43  #define WAVE_FORM_3_PATTERN  //COMPLETAR
44
45  /*=====
46  * VARIABLES GLOBALES
47  *=====*/
48  volatile uint32_t *WaveForm = (volatile uint32_t *)0x20080000;
49
50  volatile uint8_t  activeWave    = 0;
51  volatile uint8_t  indexWave     = 0;
52  volatile uint8_t  vTicks        = 0;
53  volatile uint32_t nCurrTicks     = N_TICKS_10MS;
54  volatile uint32_t cSysTickState = SYSTICK_CTRL_ON;
55
56  /*=====
57  * PROTOTIPOS DE FUNCIONES
58  *=====*/
59  static inline void cfgNVIC(int32_t, uint32_t);
60
61  void cfgGPIO(void);
62  void cfgIntExt(void);
63  void cfgIntGPIO(void);
64  void cfgSysTick(uint32_t, uint32_t);
65
66  void setWavePattern(uint8_t, uint8_t);
67  void selectNextWave(void);
68  void updateWaveAverage(void);
69

```

```

70  /*=====
71  * PROGRAMA PRINCIPAL
72  *=====*/
73  int main(void)
74  {
75      /*-----
76      * INICIALIZACIÓN DEL MCU
77      *-----*/
78      setWavePattern(WAVE_FORM_0, WAVE_FORM_0_PATTERN);
79      setWavePattern(WAVE_FORM_1, WAVE_FORM_1_PATTERN);
80      setWavePattern(WAVE_FORM_2, WAVE_FORM_2_PATTERN);
81      setWavePattern(WAVE_FORM_3, WAVE_FORM_3_PATTERN);
82
83      cfgGPIO();
84      cfgIntGPIO();
85      cfgIntExt();
86      cfgSysTick(nCurrTicks, cSysTickState);
87
88      updateWaveAverage();
89
90      /*-----
91      * INICIO PROGRAMA PRINCIPAL
92      *-----*/
93      while(1){}
94
95      return 0;
96  }
97
98  /*=====
99  * DEFINICIONES DE FUNCIONES
100  *=====*/
101  /*****
102  * @brief   Configura el NVIC y SCB para la habilitación de interrupciones.
103  *
104  * @param   <irq_number>      Número de IRQ en la tabla del vector.
105  * @param   <priority_number> Nivel de prioridad del vector.
106  *
107  * @return  None
108  *****/
109  static inline void cfgNVIC(int32_t irq_number, uint32_t priority_number)
110  {
111      //Asigna Prioridad de Interrupción
112      if(irq_number < 0) {
113          //Ejecuta interrupciones del sistema.
114          //SHP = System Handler Priority Registers
115          SCB->SHP[((uint32_t)(irq_number) & 0xF) - 4] = ((priority_number << (8 -
116          NVIC_PRIO_BITS)) & 0xFF);
117      } else {
118          //Ejecuta interrupciones de periféricos.
119          //IP = Interrupt Priority Registers
120          NVIC->IP[(uint32_t)(irq_number)] = ((priority_number << (8 - NVIC_PRIO_BITS)) &
121          0xFF);
122      }
123
124      //Habilitación de Interrupción
125      NVIC->ISER[((uint32_t)irq_number >> 5)] = (1 << ((uint32_t)irq_number & 0x1F));
126  }
127
128  /*****
129  * @brief   Escribe un patrón binario de 8 bits en el byte correspondiente
130  *          del puntero WaveForm.
131  *
132  * @param   <waveIndex>      Índice del byte a modificar (0 a 3).
133  * @param   <wavePattern>    Patrón binario de 8 bits.
134  *
135  * @return  None
136  *****/
137  void setWavePattern(uint8_t waveIndex, uint8_t wavePattern)
138  {

```

```

137     uint8_t shift = waveIndex * SIZE_WAVE;
138
139     *WaveForm &= ~(0xFF << shift);           // Limpia el byte objetivo
140     *WaveForm |= ((uint32_t)wavePattern << shift); // Carga el nuevo patrón
141 }
142
143 /*****
144  * @brief Conmuta secuencialmente a la siguiente forma de onda (0 a 3),
145  *         reinicia el índice de envío serie y actualiza el promedio en P2.
146  *
147  * @param None
148  *
149  * @return None
150  *****/
151 void selectNextWave(void)
152 {
153     activeWave = (activeWave + 1) % NUM_WAVES; // Buffer circular de ondas (0 a 3)
154     indexWave = 0;                             // Reinicia la transmisión serie
155     updateWaveAverage();                        // Actualiza la salida P2[7:0]
156 }
157
158 /*****
159  * @brief Calcula la cantidad de bits activos ('1') en la forma de onda
160  *         activa y despliega el valor acumulado en P2[7:0].
161  *
162  * @param None
163  *
164  * @return None
165  *****/
166 void updateWaveAverage(void)
167 {
168     uint8_t sum = 0;
169     uint8_t offset = activeWave * SIZE_WAVE;
170
171     for(uint8_t i = 0; i < SIZE_WAVE; i++)
172     {
173         if ((*WaveForm >> (i + offset)) & 0x01)
174         {
175             sum++;
176         }
177     }
178
179     LPC_GPIO2->FIOCLR = (0xFF << 0);
180     LPC_GPIO2->FIOSET = (sum << 0);
181 }
182
183 /*****
184  * @brief Configura las direcciones GPIO, modos de función y resistencias.
185  *
186  * @details - P0.0 a P0.7: Entradas digitales.
187  *          - P2.0 a P2.7: Salidas digitales.
188  *          - P2.8: Salida digital.
189  *          - P2.10: Función EINT0 con Pull-Down interno.
190  *          - P2.11: Función EINT1 con Pull-Up interno.
191  *
192  * @param None
193  *
194  * @return None
195  *****/
196 void cfgGPIO(void)
197 {
198     //COMPLETAR
199
200     return;
201 }
202
203 /*****
204  * @brief Configura la interrupción por GPIO en los pines P0.0 a P0.3.
205  *

```

```

206 * @details - Habilita la detección de interrupción por flanco descendente en los
207 *           pines P0.0 a P0.3.
208 *           - Limpia banderas pendientes previas, asigna la prioridad
209 *           en el NVIC y habilita el vector EINT3_IRQn.
210 *
211 * @param None
212 *
213 * @return None
214 *****/
215 void cfgIntGPIO(void)
216 {
217     //COMPLETAR
218
219     return;
220 }
221
222 /*****
223 * @brief Configura las interrupciones externas EINT0 y EINT1.
224 *
225 * @details - EINT0 (P2.10): Detección por Flanco Ascendente.
226 *           - EINT1 (P2.11): Detección por Flanco Descendente.
227 *           - Limpia banderas pendientes y asigna prioridades en el NVIC.
228 *
229 * @param None
230 *
231 * @return None
232 *****/
233 void cfgIntExt(void)
234 {
235     //COMPLETAR
236
237     return;
238 }
239
240 /*****
241 * @brief Configura el módulo SysTick Timer
242 *
243 * @param <nTicks> Valor de recarga del registro LOAD.
244 * @param <cTicks> Registro de control CTRL (Habilitación/Reloj/IRQ).
245 *
246 * @return None
247 *****/
248 void cfgSysTick(uint32_t nTicks, uint32_t cTicks)
249 {
250     //COMPLETAR
251
252     return;
253 }
254
255 /*****
256 * @brief Rutina de Servicio de Interrupción (ISR) para el vector EINT0.
257 *
258 * @details Conmuta a la siguiente forma de onda.
259 *
260 * @param None
261 *
262 * @return None
263 *****/
264 void EINT0_IRQHandler(void)
265 {
266     selectNextWave();
267
268     //COMPLETAR
269
270     return;
271 }
272
273 /*****
274 * @brief Rutina de Servicio de Interrupción (ISR) para el vector EINT1.

```

```

275 *
276 * @details Alterna período total entre 80 ms (10 ms/bit) y 160 ms (20 ms/bit)
277 *
278 * @param None
279 *
280 * @return None
281 *****/
282 void EINT1_IRQHandler(void)
283 {
284     //COMPLETAR
285
286     return;
287 }
288
289 /*****
290 * @brief Rutina de Servicio de Interrupción (ISR) para el vector SysTick.
291 *
292 * @details Genera la salida serie del bit correspondiente por P2.8.
293 *
294 * @param None
295 *
296 * @return None
297 *****/
298 void SysTick_Handler(void)
299 {
300     uint8_t currByte = (*WaveForm >> (activeWave * SIZE_WAVE)) & 0xFF;
301
302     if ((currByte >> indexWave) & 0x01)
303     {
304         LPC_GPIO2->FIOSET = (1 << 8);
305     }
306     else
307     {
308         LPC_GPIO2->FIOCLR = (1 << 8);
309     }
310
311     indexWave = (indexWave + 1) % SIZE_WAVE;
312     return;
313 }
314
315 /*****
316 * @brief Rutina de Servicio de Interrupción (ISR) para el vector EINT3.
317 *
318 * @details Máquina de estados que captura el patrón del puerto
319 *          P0[7:0] y actualiza el byte correspondiente en RAM.
320 *
321 * @param None
322 *
323 * @return None
324 *****/
325 void EINT3_IRQHandler(void)
326 {
327     static uint8_t waveState = WAVE_FORM_0;
328
329     uint8_t capPattern = (uint8_t)(LPC_GPIO0->FIOPIN & 0xFF);
330
331     setWavePattern(waveState, capPattern);
332
333     waveState = (waveState + 1) % NUM_WAVES;
334
335     //COMPLETAR
336 }

```